

1.1 为什么需要机器人操作系统

所属阶段： 第一阶段 · ROS 2 入门与环境搭建

讲次数量： 6 节

预计时长： ~35 分钟

前置依赖： 无

学习目标

完成本节后，学员将能够：

- 理解自动驾驶与机器人时代的技术背景与产业机遇
- 描述工业机器人、服务机器人和自动驾驶汽车的发展现状
- 识别机器人软件开发中的三大核心挑战：硬件异构性、模块复用、团队协作
- 解释 ROS 2 硬件抽象如何实现"即插即用"的模块化开发
- 对比 ROS 1 与 ROS 2 的设计差异，说明为何需要全新的机器人操作系统
- 掌握 ROS 2 的四大核心设计原则：DDS 通信、实时性、安全性、跨平台

讲次 1：自动驾驶与机器人时代的背景与机遇

知识点

机器人学（Robotics）是一门研究如何让机器替代或辅助人类完成物理与认知任务的学科。从学科定义来看，机器人需要同时具备两类能力：

1. 物理活动能力（Physical Activities）

包括抓取、搬运、装配、行走、驾驶等，需要机械结构、执行器（电机、液压、气动）和传感器（力觉、视觉、激光雷达）的协同工作。

2. 决策能力（Decision-Making）

包括路径规划、障碍物避让、目标识别、任务调度等，需要算法、感知融合和控制系统支撑。

过去二十年，计算能力、传感器成本和人工智能技术的进步，使这两类能力逐步从实验室走向商业应用。不同领域的成熟度并不相同：工业机器人已规模部署，服务机器人在限定场景快速迭代，自动驾驶则必须按法规等级和运行设计域（ODD）判断能力。

- **自动驾驶汽车**不再是科幻概念。Waymo、百度萝卜快跑等在限定区域运营 L4 Robotaxi；Tesla FSD、小鹏 XNGP 等量产系统仍属于需要驾驶员持续监督的 L2/L2+；奔驰 Drive Pilot 等系统在获批条件下提供 L3。
- **工业机器人**从汽车产线扩展到 3C 电子、物流仓储、食品包装等领域，协作机器人（Cobot）让人机共融成为可能。
- **服务机器人**进入酒店、餐厅、医院、商场，配送和清洁机器人已在部分场所常态化运营。
- **特种机器人**在搜救、巡检、农业、深海探测等场景发挥不可替代的作用。

无论哪类机器人，它们的软件架构都遵循一个共同的底层模式：**感知 → 决策 → 执行**。

首次出现的术语：LiDAR 是“激光雷达”，通过测距形成环境轮廓或点云；IMU 是“惯性测量单元”，测量角速度和加速度；GNSS 是“全球卫星导航系统”；SLAM 是“同步定位与建图”。后续章节会分别展开，本节只需先理解它们在感知与定位链路中的位置。

👁️ 感知层（Perception）

激光雷达 · 摄像头 · IMU · 毫米波雷达 · GNSS

输出：点云、图像、惯性数据 → 传感器融合 → 环境感知

sensor_msgs

🧠 决策层（Planning）

SLAM 建图定位 · 目标检测与识别 · 路径规划 · 行为决策

输出：全局/局部路径、速度/角速度指令 → 发送给执行层

geometry_msgs/Twist

⚙️ 执行层（Control / Actuation）

运动控制 · 机械操作 · 底盘驱动 · 关节伺服

输出：轮速 PWM、关节角度 → 物理世界交互 → 改变环境状态

图 1.1：机器人通用软件架构 —— 感知、决策、执行的闭环

示例：自动驾驶汽车通过激光雷达和摄像头感知周围环境（感知层），SLAM 算法构建地图并定位车辆位置，规划模块计算最优路径（决策层），控制模块将路径转化为方向盘转向和油门/刹车指令（执行层）。这三个环节之间需要高效的数据传递和协调，这正是 ROS 2 要解决的核心问题。

对开发者而言，这意味着巨大的职业机遇：机器人软件工程师、自动驾驶感知工程师、SLAM 算法工程师、机器人系统集成工程师等岗位需求持续增长。但机遇背后是对**系统化工程能力**的要求——你不仅需要会写算法，还需要理解硬件、通信、仿真、部署的完整链路。这正是 ROS 2 这类机器人操作系统存在的根本原因。

关键点

- 不同机器人共享“感知 → 决策 → 执行 → 反馈”的闭环，但商业成熟度与安全边界不同
 - ROS 2 解决的是跨硬件、跨模块和跨团队的系统集成问题，不代替具体算法或安全设计
-

讲次 2：工业机器人、服务机器人、自动驾驶汽车的发展现状

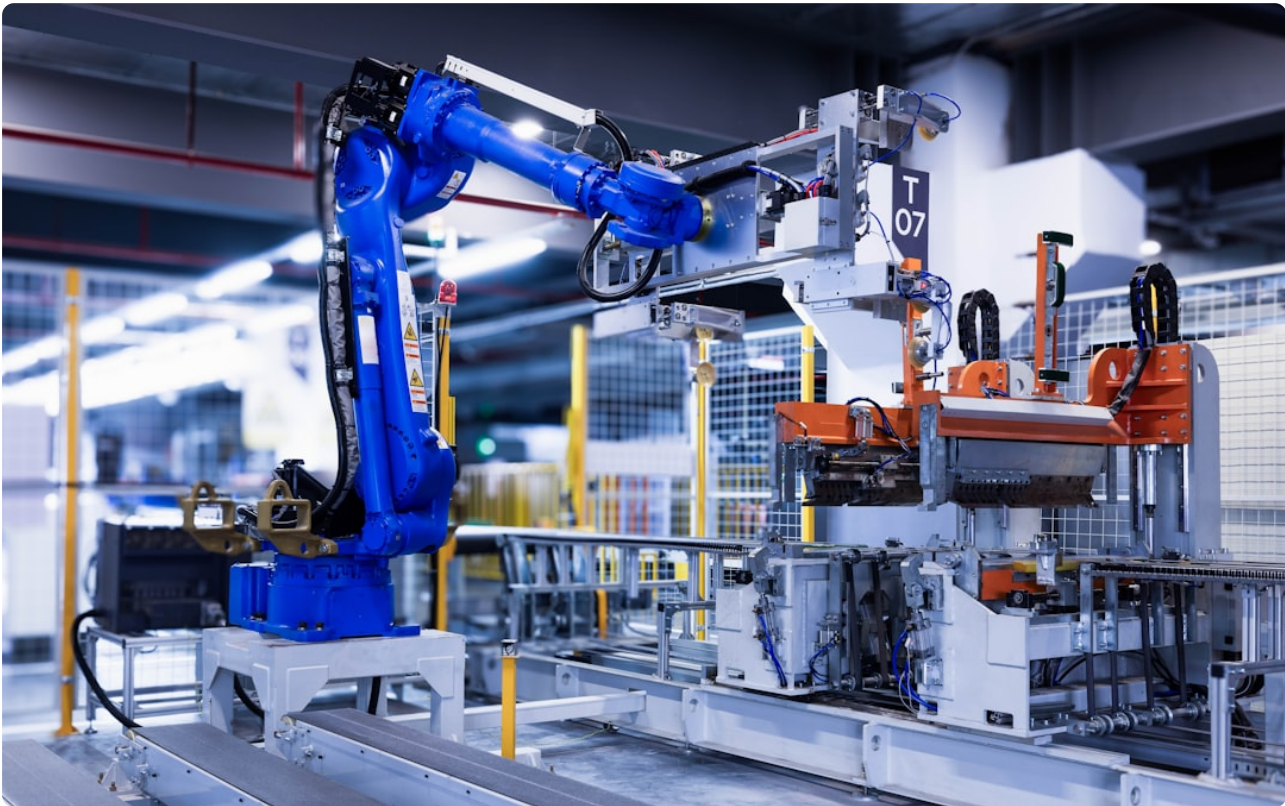
知识点

工业机器人

工业机器人是最早大规模商业化的机器人形态，主要部署在制造工厂：

类型	典型应用	代表厂商
多关节机械臂	焊接、喷涂、装配	ABB、KUKA、Fanuc、UR
SCARA 机器人	电子元件贴装、搬运	Epson、汇川
Delta 并联机器人	高速分拣、包装	ABB、勃肯特
协作机器人（Cobot）	人机协作装配、检测	Universal Robots、越疆、节卡

发展趋势：从隔离式工业机械臂扩展到满足风险评估和安全规范的协作机器人；从固定工位走向移动操作（Mobile Manipulation）。ROS/ROS 2 常用于科研、原型、系统集成和部分产品，但传统 PLC、厂商控制器与自研中间件仍大量存在。





服务机器人

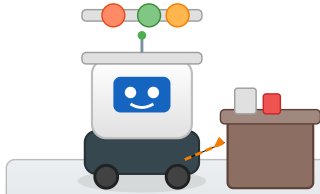
服务机器人面向非工业场景，形态更加多样：

- **室内配送机器人**：酒店送物、餐厅送餐（普渡、擎朗、云迹）
- **清洁机器人**：商场、机场地面清洁（高仙、科沃斯商用）
- **导览/迎宾机器人**：博物馆、展厅接待（优必选、猎户星空）
- **医疗辅助机器人**：手术辅助、康复训练、药品配送
- **农业机器人**：采摘、喷洒、巡检

服务机器人的核心挑战是**非结构化环境**：人流密集、地形多变、任务多样。这要求软件架构具备高度的模块化和可扩展性。

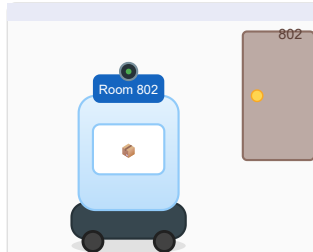
服务机器人应用场景

餐厅配送机器人



自主导航配送菜品到餐桌
普渡 / 擎朗 / Keenon

酒店/医院配送机器人



自主乘电梯、送物到客房
云迹 / 优地 / Relay

商用清洁机器人



商场/机场/医院自主清洁
高仙 / Avidbots / Brain Corp

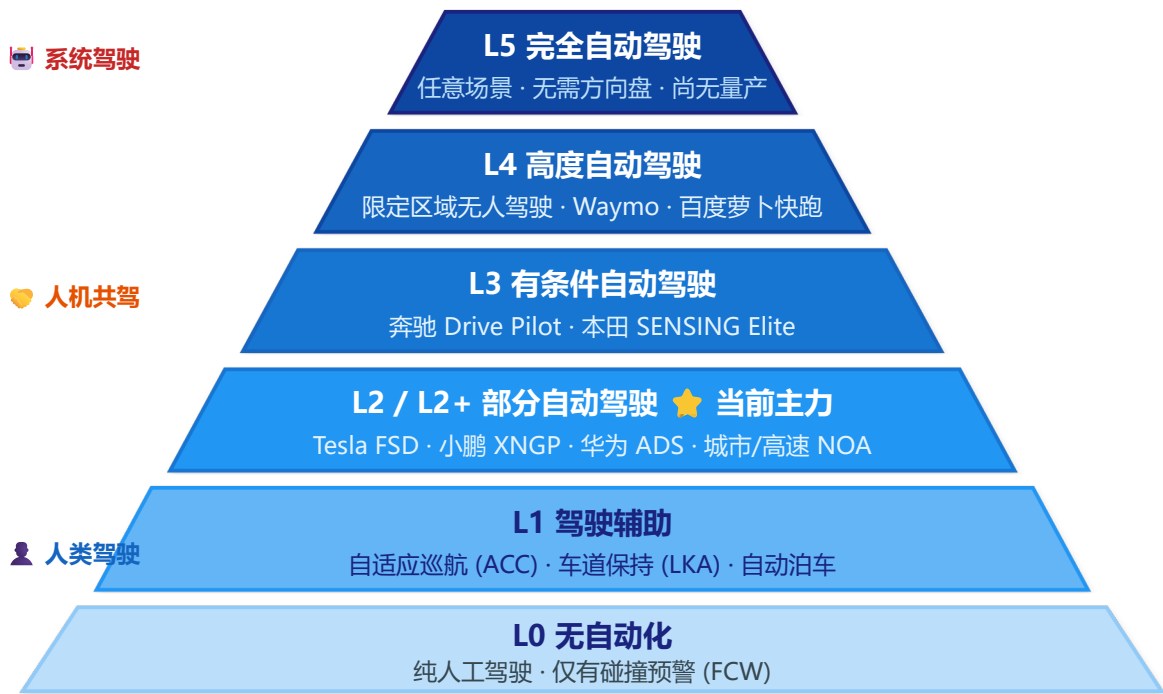
服务机器人核心挑战：在非结构化环境中自主感知、决策、导航

自动驾驶汽车

自动驾驶按 SAE 标准分为 L0-L5 六个等级。当前商业化主要集中在 L2+（高级辅助驾驶）和 L4（限定区域无人驾驶）：

- **L2+**：高速 NOA、城市 NOA，人类驾驶员仍需监控
- **L4 Robotaxi**：Waymo（凤凰城、旧金山）、百度萝卜快跑（武汉、北京）
- **L4 物流车**：主线科技、赢彻科技干线物流

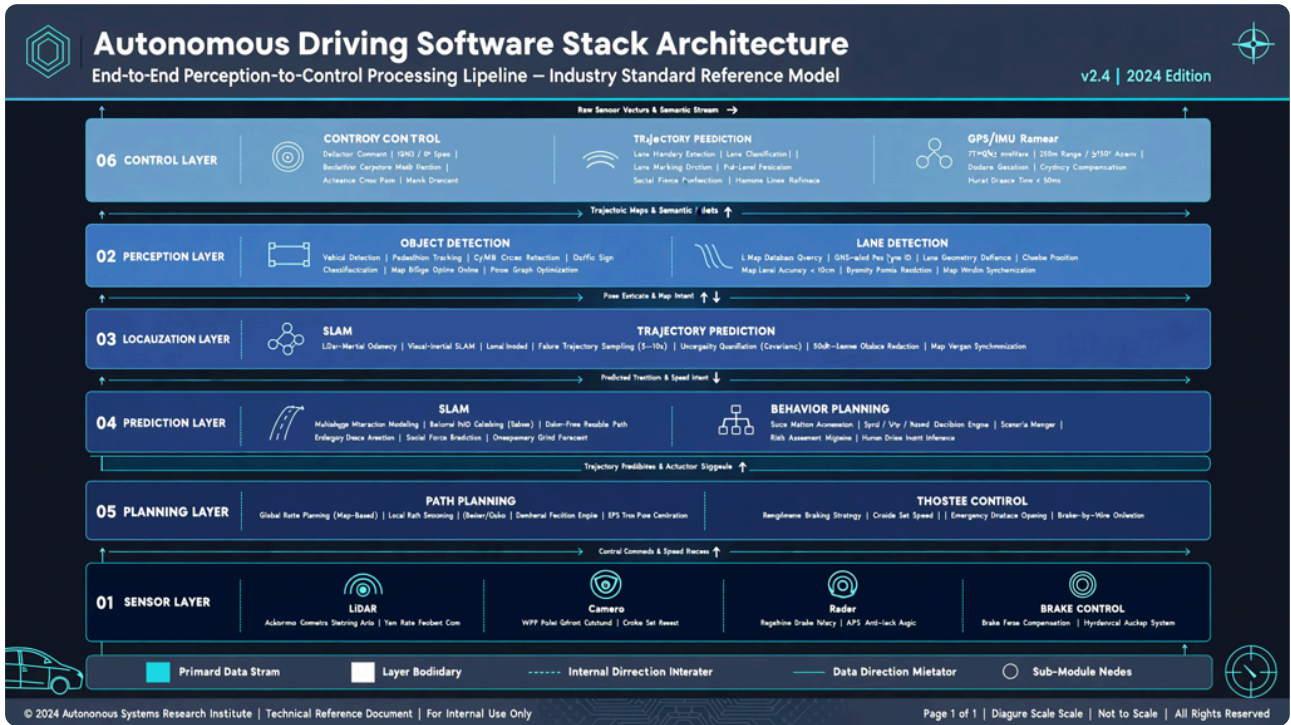
SAE J3016 自动驾驶分级标准



当前商业化前沿：L2+ 城市 NOA（量产主力）· L4 Robotaxi（限定城市运营）
ROS 2 应用场景：原型开发 · 仿真验证 · 传感器融合 · SLAM 算法研发

图 1.2: SAE 自动驾驶等级金字塔 (L0-L5)

自动驾驶软件栈通常包含：感知（相机+激光雷达+毫米波雷达）→ 定位（GNSS+IMU+SLAM）→ 预测 → 规划 → 控制。各模块之间需要高效的通信机制，这正是 ROS 2 擅长的领域。





三类机器人应用场景多维对比



图 1.3：三类机器人应用场景多维对比 —— 工业、服务、自动驾驶

三者的共同需求

无论是工业臂、服务机器人还是自动驾驶汽车，软件开发都面临相似的底层需求：

- 多传感器数据融合与同步
- 模块化算法组件的集成
- 仿真验证与实车/实机部署的一致性
- 跨平台、跨硬件的可移植性

这些共性需求催生了对统一机器人操作系统的强烈诉求。

关键点

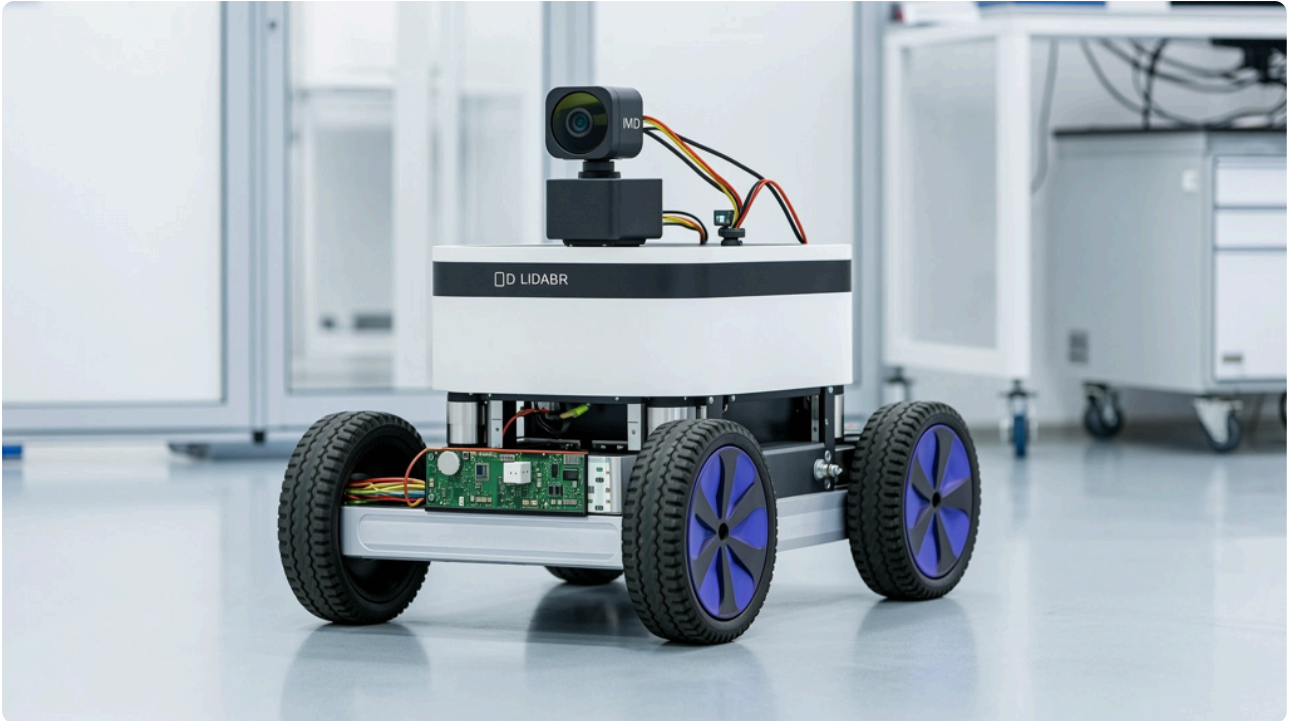
- 工业、服务和自动驾驶的环境结构化程度、安全要求及量产软件栈不同，不能用一个案例替代全部行业

- 三类系统的共同工程需求是数据同步、模块集成、仿真验证和跨硬件迁移
-

讲次 3：机器人软件开发面临的核心挑战

知识点

假设你要从零开发一台移动机器人（带激光雷达、摄像头、IMU、轮式底盘），你会立刻遇到以下挑战：



挑战一：硬件异构性 (Hardware Heterogeneity)

机器人硬件组合千变万化：

- **传感器**：激光雷达有 Velodyne、Hokuyo、RPLidar 等数十个品牌，接口有 USB、以太网、串口
- **执行器**：直流电机、步进电机、伺服电机，驱动板有 STM32、Arduino、树莓派
- **计算平台**：x86 工控机、ARM 嵌入式板、NVIDIA Jetson、Qualcomm RB5

每种硬件有自己的 SDK、通信协议和数据格式。如果每个项目都从零对接硬件，开发周期将被无限拉长。



挑战二：模块复用 (Module Reusability)

机器人软件包含大量通用模块：

- SLAM 建图算法 (Cartographer、SLAM Toolbox)
- 路径规划 (Nav2、MoveIt 2)
- 视觉检测 (OpenCV、YOLO 集成)
- 坐标变换 (TF2)

这些模块经过大量测试和优化，不应该每个项目都重新实现。理想状态是像搭积木一样，将成熟模块组合成完整系统。

挑战三：团队协作 (Team Collaboration)

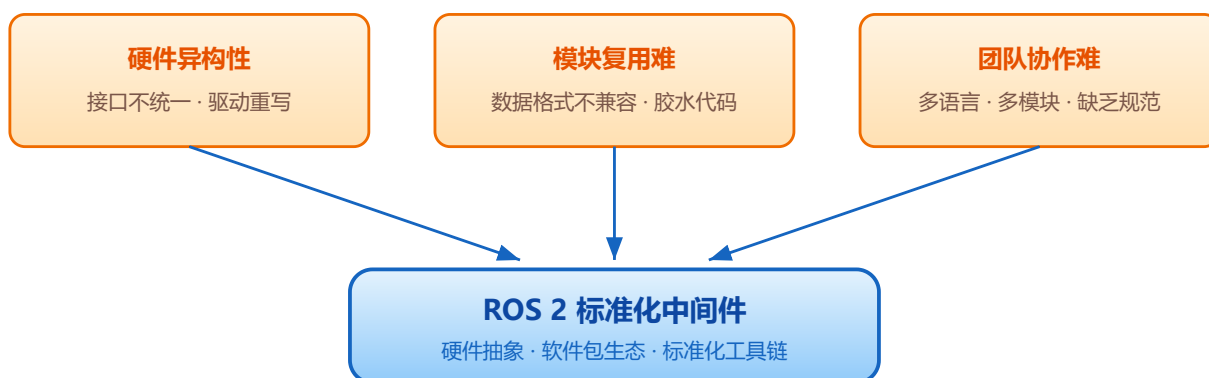
一个机器人项目通常需要多人协作：

- 硬件工程师负责电路和机械
- 感知工程师负责传感器驱动和目标检测
- 规划工程师负责路径规划和控制
- 系统集成工程师负责整体联调

如果没有统一的通信标准和开发框架，各模块之间的接口定义、数据格式、时序同步将成为协作瓶颈。

三大挑战共同指向同一个需求——**标准化中间件**，而 ROS 2 正是为此而生：

三大挑战 → 标准化中间件需求



如果驱动、消息格式和通信层都与业务代码绑定，每换一次硬件或项目就会重复适配。三大挑战因此共同指向标准化中间件，而不是要求团队重新实现一套基础设施。

讲次 4：从"重写一切"到"即插即用"——ROS 2 硬件抽象的价值

知识点

ROS 的诞生

2007 年，Willow Garage 的工程师们意识到机器人软件开发需要一个统一的框架。2009 年，**ROS (Robot Operating System)** 首次发布，其核心理念是：

提供一套标准化的通信机制、工具链和软件包生态，让开发者专注于算法本身，而非底层基础设施。

2013 年，Open Source Robotics Foundation (OSRF, 后更名为 Open Robotics) 接管 ROS 的维护和发展。如今，ROS 已广泛用于机器人研究、原型开发和产品集成。

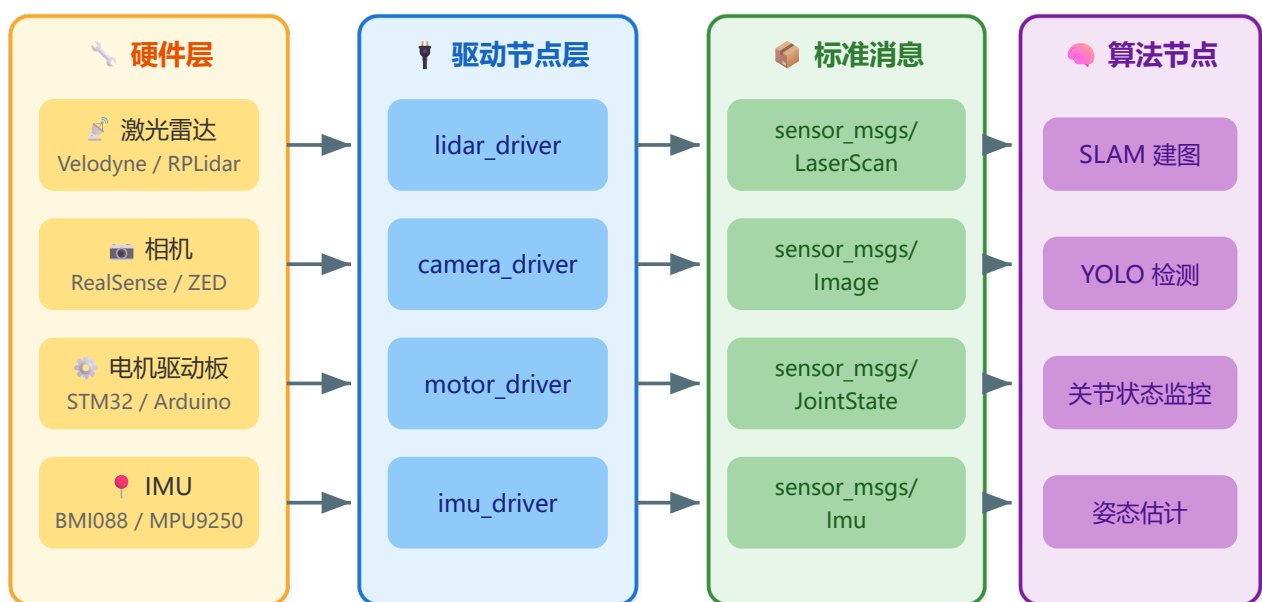
ROS 如何解决三大挑战

传统开发把硬件 SDK 直接耦合进算法，ROS 2 则用驱动节点和标准消息缩小硬件替换的影响范围。下面用具体数据链说明这种抽象如何工作。

1. 硬件抽象层 (Hardware Abstraction)

ROS 2 通过**驱动节点 (Driver Node)** 将硬件细节封装在底层，向上层提供标准化的消息接口。

⚠ **概念辨析：**这里的"驱动节点"是运行在**用户空间**的 ROS 2 节点（通常用 C++ 编写），它通过硬件厂商提供的 SDK 或 Linux 设备接口（如 `/dev/ttyUSB0`、网络 socket）与硬件通信。这与 Linux **内核驱动**（用 C 编写，运行在内核空间）是不同的层次。完整的调用链是：硬件 → Linux 内核驱动(C) → 设备接口 → 厂商 SDK(C/C++) → ROS 2 驱动节点(C++) → ROS 2 话题。



图：ROS 2 硬件抽象层 —— 驱动节点将硬件封装为标准消息，上层算法即插即用

图中前三条是“硬件状态向上发布”的简化数据流。控制命令方向相反，例如 Nav2 发布 `geometry_msgs/Twist`，底盘控制节点将其转换为轮速或总线指令。真实机器人通常同时存在状态上行与命令下行，不能把所有硬件接口都理解为单向传感器数据。

上层算法节点通过标准消息与驱动解耦，不必直接调用每款硬件的 SDK。理想情况下，更换硬件只需替换驱动和配置；真实项目仍必须核对话题名、坐标系（TF）、QoS、单位、量程、时间戳与外参标定。这里的“即插即用”指**缩小修改范围**，不是承诺任何硬件都能零配置替换。

2. 软件包生态 (Package Ecosystem)

ROS 社区积累了数千个开源软件包 (Package)，覆盖：

- 传感器驱动 (`velodyne_driver`、`realsense2_camera`)
- SLAM 算法 (`slam_toolbox`、`cartographer`)
- 导航框架 (`nav2_bringup`)
- 机械臂控制 (`moveit2`)
- 仿真环境 (`ros_gz`，Jazzy 使用 Gazebo Harmonic)

通过 `apt install ros-jazzy-<package>` 或从源码编译，即可将这些成熟模块集成到你的项目中。

3. 标准化工具链

ROS 2 提供了一整套开发和调试工具：

工具	用途
<code>ros2</code> CLI	命令行查看节点、话题、服务
RViz 2	3D 可视化
Gazebo	物理仿真
rqt	图形化调试面板
colcon	构建系统
rosdep	依赖管理

这些工具大幅降低了团队协作的沟通成本。

类比理解

可以把 ROS 2 类比为机器人领域的“操作系统”，但要注意它通常运行在 Linux/Windows 等宿主操作系统之上，并不是内核：

- 就像 Windows/macOS 为 PC 应用提供文件系统、网络、图形界面等基础设施
- ROS 2 为机器人应用提供通信、坐标变换、可视化、仿真等基础设施
- 开发者像组合应用组件一样编写 ROS 2 节点 (Node)；ROS 2 负责通信与工具链，进程、线程、内存和设备资源仍由宿主操作系统管理

关键点

- ROS 2 是运行在宿主操作系统之上的通信框架、软件包生态和工具链，不是 Linux/Windows 内核的替代品

- 硬件抽象能缩小改动范围，但换硬件仍要复核 Topic、TF、QoS、单位、时间戳和标定
-

讲次 5：ROS 2 vs ROS 1——为什么需要一个全新的机器人操作系统

知识点

ROS 1 在十年间（2009–2019）取得了巨大成功，但随着机器人应用场景的扩展，其架构局限性日益凸显。Open Robotics 基于多年积累的经验，于 2014 年启动 ROS 2 的设计工作，2017 年 12 月发布第一个正式版 Ardent。

ROS 1 的设计假设与局限

ROS 1 最初为**实验室环境中的单台有线网络机器人**设计：

ROS 1 假设	现实变化	导致的问题
实验室内的单机器人	多机器人协作、蜂群	中央 Master 成为发现与注册的单点依赖
可靠局域网	无线、跨网段通信	缺少细粒度 QoS，弱网适配能力有限
信任环境	安全敏感场景	无认证、无加密
不以确定性实时为首要目标	工业控制、自动驾驶	默认软件栈难以给出端到端时延保证
主要面向 Ubuntu Linux	嵌入式、Windows、RTOS	其他平台支持与工具链不统一

ROS 1 的核心痛点：

- 中央 Master 节点 (roscore)**
节点需要向 Master 注册并查询连接信息。Master 故障后，已经建立的部分点对点连接可能继续传输，但新节点发现、重连和名称查询会受阻，因此它仍是系统级单点依赖。
- 自定义通信协议 (TCPROS/UDPROS)**
TCPROS/UDPROS 能完成常规通信，也支持少量传输选项，但缺少 ROS 2 那样统一、细粒度的可靠性、历史深度、截止时间和持久性 QoS 模型。
- 缺乏安全机制**
任何能访问网络的人都可以订阅/发布话题，无法用于医疗、国防等安全敏感场景。
- 历史兼容负担** ROS 1 长期依赖 Python 2；最后一个发行版 Noetic 已迁移到 Python 3，但整体架构仍需兼容大量历史包和工具。
- 跨平台与工程化能力有限** catkin/CMake 本身可以管理大型项目，但 ROS 1 的跨平台、隔离构建和现代部署体验不如 ROS 2 统一。

ROS 2 的改进

ROS 2 不是 ROS 1 的简单升级，而是**从零重新设计**：

从 ROS 1 到 ROS 2：不是升级，而是重新设计



维度	ROS 1	ROS 2
通信中间件	自研 TCPROS + Master	DDS (Data Distribution Service) 标准
节点发现	中央 Master	分布式自动发现 (DDS Discovery)
QoS 策略	缺少 ROS 2 式统一、细粒度策略模型	支持 Best Effort / Reliable 等多种策略
安全性	默认通信协议缺少统一认证、加密和访问控制	DDS Security (认证、加密、访问控制)
实时性	默认不提供端到端确定性保证	提供实时友好 API，可结合 PREEMPT_RT 与实时工程方法
跨平台	主要 Ubuntu Linux	按发行版 Tier 支持 Ubuntu / Windows / RHEL 等平台
Python	早期版本以 Python 2 为主，Noetic 使用 Python 3	Python 3
构建系统	catkin	colcon + ament

迁移现状

- ROS 1 已于 2025 年 5 月 31 日正式 EOL (Noetic 是最后一个发行版)
- 新项目应直接使用 ROS 2
- 主流机器人厂商 (TurtleBot、Fetch、Franka 等) 均已提供 ROS 2 支持
- 本课程基于 **ROS 2 Jazzy** (仍受支持的 LTS，支持至 2029 年) 教学；2026 年最新 LTS 是 Lyrical，本课程不因版本更新而跳过完整回归验证

ROS 1 的中央发现依赖、QoS 与安全能力、跨平台工程化和维护状态都不适合新项目继续扩展。ROS 2 是重新设计而非简单改名；本课程选择 Jazzy 是因为 Stage1–8 已在该基线上验证，而不是因为它仍是最新发行版。

讲次 6：ROS 2 的核心设计原则

知识点

ROS 2 的设计围绕四个核心原则展开，每个原则都直接对应 ROS 1 时代的痛点。

原则一：DDS 通信 (Data Distribution Service)

ROS 2 采用 **DDS** 作为底层通信中间件。DDS 是 OMG (Object Management Group) 制定的工业标准，广泛应用于国防、航空、自动驾驶等领域。

DDS 为 ROS 2 带来的能力：

- **发布/订阅通信**：节点之间通过 Topic 异步传递消息
- **服务质量 (QoS)**：可为不同话题配置可靠性、持久性、历史深度、截止时间等策略
- 高频传感器数据常用 `Best Effort`，优先保证新鲜度并容忍少量丢帧
- 状态、配置或任务类消息常用 `Reliable`；安全关键控制不能只依赖“可靠传输”，还需要超时、看门狗、急停和失效安全设计
- **自动发现**：节点启动后自动在网络中发现其他节点，无需中央 Master
- **多播与单播**：支持 LAN 内的高效数据传输

ROS 2 中的 DDS 实现 (RMW 层可切换)：

RMW 实现	DDS 底层	说明
<code>rmw_fastrtps_cpp</code>	eProsima Fast DDS	Jazzy 默认实现，性能优秀
<code>rmw_cyclonedds_cpp</code>	Eclipse Cyclone DDS	开源实现，配置和网络行为清晰，常用于 Linux 机器人系统
<code>rmw_connextdds</code>	RTI Connext DDS	商业实现，功能与授权条件以 RTI 当前版本为准

在应用层，你使用 `rclpy` 或 `rclcpp` 编写代码，完全不需要直接接触 DDS API——ROS 2 的 RMW (ROS Middleware) 层完成了抽象。

原则二：实时性 (Real-Time Support)

许多机器人场景对通信延迟有严格要求：

- 部分工业伺服控制回路：周期可达到 1 ms 或更短，但通常由专用控制器执行
- 自动驾驶紧急制动：要求确定性响应
- 多关节协调运动：要求同步精度

ROS 2 通过以下机制支持实时性：

- **可控的内存与消息路径**：可配合预分配、loaned message 等机制减少运行时分配和复制，但需要所选客户端库与 RMW 支持
- **执行器 (Executor) 模型**：提供单线程、多线程和回调组等调度结构；严格优先级与确定性仍需专门设计和测量

- **与 PREEMPT_RT Linux 内核配合**：可实现确定性/近硬实时行为（需系统级集成与验证）
- **共享内存传输**：在支持的 RMW 实现和配置下，DDS 可使用共享内存降低拷贝开销（并非默认对所有通信自动启用）

原则三：安全性 (Security)

ROS 2 集成了 **DDS Security** 插件，提供：

- **身份认证 (Authentication)**：节点必须持有有效证书才能加入网络
- **访问控制 (Access Control)**：基于权限文件限制节点能发布/订阅哪些话题
- **加密 (Encryption)**：话题数据在传输过程中加密

启用安全性的配置示例（概念层面）：

```
# 设置安全配置文件路径
export ROS_SECURITY_KEYSTORE=/path/to/keystore
export ROS_SECURITY_ENABLE=true
export ROS_SECURITY_STRATEGY=Enforce
```

安全性在 ROS 2 中是可选功能——开发阶段可以关闭以简化调试，部署到生产环境时启用。

原则四：跨平台 (Cross-Platform)

ROS 2 的支持平台按发行版和 Tier（支持等级）划分。以本课程固定使用的 Jazzy 为例：

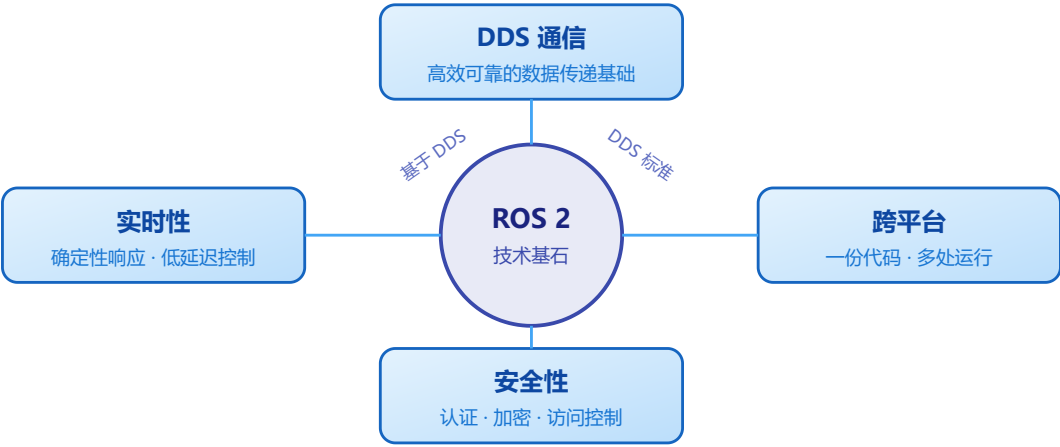
平台	支持状态	典型场景
Ubuntu 24.04 (Noble)	Tier 1	本课程开发、仿真和二进制安装主平台
Windows 10	Tier 1	原生 Windows 二进制与企业开发；本课程 Windows 用户主要走 WSL2
RHEL 9	Tier 2	企业 Linux 部署
Ubuntu 22.04 / macOS	Tier 3 或源码支持	不作为 Jazzy 课程主路径
FreeRTOS 等 RTOS + micro-ROS	独立的 micro-ROS 生态	MCU 端与 ROS 2 系统通信，不等同于在 MCU 上运行完整桌面 ROS 2

跨平台意味着：

- 遵循可移植 API 且依赖可用时，同一份 ROS 2 业务代码可以在 Ubuntu 开发机调试，再为 ARM 嵌入式 Linux 重新构建部署
- 团队可以使用不同操作系统的开发者协作
- 仿真 (Ubuntu + Gazebo) 与实机 (嵌入式 Linux) 共享同一套软件包

四大原则的关系

ROS 2 四大核心设计原则的关系



关键点

- QoS、Executor、PREEMPT_RT 和 DDS Security 提供的是工程能力，不会自动得到可靠、实时或安全的整机系统
 - 跨平台仍受发行版 Tier、CPU 架构、驱动和第三方依赖约束，通常需要重新构建与回归测试
-

本节总结

核心概念回顾

概念	要点
机器人时代	物理执行 + 智能决策，多行业规模化落地
三大挑战	硬件异构性、模块复用、团队协作
ROS 价值	硬件抽象、软件包生态、标准化工具链
ROS 1 → ROS 2	从中央发现依赖转向 RMW/DDS 等分布式通信，并强化 QoS、安全、实时工程与跨平台能力
四大原则	DDS 通信、实时性、安全性、跨平台

下一步

下一节 **1.2 ROS 2 架构概览** 中，我们将深入 ROS 2 的内部分层架构，了解从应用层到 DDS 中间件的完整技术栈，以及 Python 与 C++ 客户端库的选择策略和发行版规划。